

1. Estructura del repositorio

Mantendremos un solo repositorio Git:

Proyecto Manuel Diaz/

```
|— frontend/  
|— backend/  
|— .gitignore  
|— README.md
```

Frontend y backend seguirán siendo proyectos independientes:

frontend/package.json

backend/package.json

Esto permite ejecutarlos por separado durante el desarrollo, pero desplegarlos juntos desde el mismo repositorio.

2. Estructura interna del backend

Recomiendo organizarlo por funcionalidad:

```
backend/  
|— src/  
|   |— app.js  
|   |— server.js  
|   |  
|   |— config/  
|   |   |— env.js  
|   |  
|   |— db/  
|   |   |— pool.js  
|   |   |— transaction.js  
|   |   |— migrations/  
|   |  
|   |— middleware/  
|   |   |— authenticate.js  
|   |   |— authorizeAdmin.js  
|   |   |— errorHandler.js  
|   |   |— notFound.js  
|   |   |— originGuard.js  
|   |   |— rateLimiters.js
```

```
|
|
|   └─ validate.js
|
| └─ modules/
|   └─ auth/
|     └─ auth.routes.js
|     └─ auth.controller.js
|     └─ auth.service.js
|     └─ auth.repository.js
|     └─ auth.schemas.js
|
|   └─ users/
|   └─ matterTypes/
|   └─ matters/
|
| └─ shared/
|   └─ AppError.js
|   └─ asyncHandler.js
|   └─ pagination.js
|
| └─ constants/
|   └─ matterStatuses.js
|   └─ documentStatuses.js
|   └─ permissions.js
|
| └─ scripts/
|   └─ createAdmin.js
|   └─ seedDevelopment.js
|
| └─ tests/
|   └─ unit/
|   └─ integration/
|   └─ helpers/
|
| └─ .env.example
| └─ .gitignore
| └─ eslint.config.js
| └─ package.json
| └─ README.md
```

Responsabilidad de los archivos principales

- `server.js`: inicia el servidor y escucha el puerto.

- `app.js`: configura Express, seguridad, rutas y errores.
- `env.js`: valida las variables de entorno.
- `pool.js`: mantiene las conexiones con PostgreSQL.
- `transaction.js`: facilita operaciones que deben completarse juntas.
- `routes`: declara rutas y permisos.
- `controller`: traduce HTTP a operaciones del sistema.
- `service`: contiene las reglas del negocio.
- `repository`: ejecuta consultas PostgreSQL.
- `schemas`: valida los datos recibidos.

El flujo será:

Request

- Route
- Authentication
- Validation
- Controller
- Service
- Repository
- PostgreSQL
- Response

3. Dependencias del backend

Dependencias de ejecución

Paquete	Uso
<code>express</code>	API y servidor
<code>pg</code>	Conexión con PostgreSQL
<code>zod</code>	Validación
<code>argon2</code>	Hash de contraseñas
<code>helmet</code>	Cabeceras de seguridad
<code>cors</code>	Desarrollo con frontend separado
<code>cookie-parser</code>	Lectura de la cookie de sesión
<code>express-rate-limit</code>	Protección del login y API
<code>pino</code>	Logs estructurados
<code>pino-http</code>	Logs de solicitudes
<code>dotenv</code>	Variables locales
<code>node-pg-migrate</code>	Migraciones

Dependencias de desarrollo

Paquete	Uso
supertest	Pruebas HTTP
eslint	Revisión del código
globals	Entorno Node para ESLint

No necesitaremos:

- Prisma
- Sequelize
- Mongoose
- Passport
- JSON Web Tokens
- Nodemon
- TypeScript inicialmente

Node 24 ya incluye modo `--watch` y ejecutor de pruebas, por lo que evitaremos dependencias innecesarias.

4. Scripts previstos

El backend tendrá comandos conceptualmente equivalentes a:

```
npm run dev
npm start
npm test
npm run lint
npm run migrate
npm run migrate:down
npm run create-admin
npm run seed:development
```

- `dev`: servidor con reinicio automático.
- `start`: ejecución de producción.
- `test`: pruebas.
- `lint`: revisión estática.
- `migrate`: aplica migraciones pendientes.
- `migrate:down`: revierte la última migración en desarrollo.
- `create-admin`: crea el administrador inicial.
- `seed:development`: añade información ficticia, nunca datos de producción.

5. Variables de entorno

El archivo real `.env` nunca se subirá a Git.

`.env.example` documentará:

```
NODE_ENV=development
PORT=3000
```

```
DATABASE_URL=
MIGRATION_DATABASE_URL=
```

```
FRONTEND_ORIGIN=http://localhost:5173
SESSION_COOKIE_NAME=gestor_session
SESSION_TTL_HOURS=12
```

```
DATABASE_POOL_MAX=5
LOG_LEVEL=info
```

Diferencia entre las conexiones

`DATABASE_URL`:

- Utilizada por Express.
- En producción apuntará al Session Pooler de Supabase.
- Mantendrá un grupo pequeño de conexiones.

`MIGRATION_DATABASE_URL`:

- Utilizada únicamente para migraciones.
- Preferiblemente será la conexión directa de PostgreSQL.
- Si la red local no soporta IPv6, podremos usar temporalmente Session Pooler.

Ninguna de estas variables se añadirá al frontend.

6. Orden de migraciones

Cada migración tendrá una única responsabilidad:

```
001_create_app_schema
002_create_users
003_create_auth_sessions
```

004_create_matter_types
005_create_template_sections
006_create_template_documents
007_create_matters
008_create_matter_documents
009_create_matter_status_history
010_create_indexes_and_database_functions

001_create_app_schema

Crearé:

- Esquema privado app.
- Configuración básica.
- Función para actualizar updated_at.
- Funciones auxiliares estrictamente necesarias.

Migraciones 002 a 009

Cada una crea una tabla, sus relaciones y restricciones.

010_create_indexes_and_database_functions

Incluye:

- Índices de búsqueda.
- Índices parciales para elementos activos.
- Restricciones de unicidad sin distinguir mayúsculas.
- Funciones que no pertenecen exclusivamente a una tabla.

Las migraciones no crearán usuarios de ejemplo ni templates ficticios.

7. Administrador inicial

El administrador no aparecerá escrito en una migración ni en Git.

Usaremos:

```
npm run create-admin
```

El proceso solicitará:

Username
Full name
Password
Password confirmation

Después:

1. Validará el username.
2. Validará la contraseña.
3. Generará el hash Argon2id.
4. Creará el usuario con `system_role = ADMIN`.
5. Rechazará la operación si ya existe un administrador.

La contraseña no se mostrará al escribirla ni se guardará en logs.

8. Entornos de base de datos

Recomiendo esta separación:

Desarrollo y pruebas → PostgreSQL local
Producción → Supabase

PostgreSQL local

Ventajas:

- No consume límites gratuitos.
- Funciona sin internet.
- Podemos borrar y reconstruir datos de prueba.
- Las pruebas no ponen en riesgo datos reales.
- Aprenderás PostgreSQL directamente.

Podemos utilizar:

- Instalación local de PostgreSQL en Windows, o
- PostgreSQL mediante Docker.

Para tu situación recomendaría instalación local si todavía no utilizas Docker. Introducir Docker y PostgreSQL simultáneamente añade complejidad que no necesitamos.

Antes de crear el backend tendremos que confirmar si ya tienes PostgreSQL instalado.

9. Seguridad inicial

Desde el primer bloque de código tendremos:

- Consultas parametrizadas.
- Validación con Zod.
- Contraseñas con Argon2id.
- Cookies HttpOnly.
- Rate limit específico para login.
- Rate limit general más permisivo.
- CORS limitado al frontend conocido.
- Verificación de Origin en operaciones mutables.
- Cabeceras Helmet.
- Límite de tamaño para JSON.
- Errores internos sin detalles sensibles.
- Logs sin contraseñas, cookies ni tokens.
- Esquema PostgreSQL privado.
- Permisos administrativos comprobados en el backend.

El frontend podrá esconder botones administrativos, pero la protección real estará en Express.

10. Respuestas de la API

Mantendremos una estructura consistente.

Éxito:

```
{
  "data": {},
  "message": "Matter created successfully."
}
```

Error de validación:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid data.",
    "fields": {
      "matterNumber": "Enter exactly 6 digits."
    }
  }
}
```

```
}  
}
```

Conflicto de concurrencia:

```
{  
  "error": {  
    "code": "VERSION_CONFLICT",  
    "message": "This matter was modified by another user."  
  }  
}
```

No enviaremos trazas, SQL ni detalles internos al navegador.

11. Paginación

La ruta de matters aceptará:

GET /api/matters?page=1&pageSize=25

También:

search
status
matterTypeId
sort
direction

Respuesta:

```
{  
  "data": [],  
  "pagination": {  
    "page": 1,  
    "pageSize": 25,  
    "totalItems": 10000,  
    "totalPages": 400  
  }  
}
```

Para 10.000 matters, paginación tradicional es suficiente y más sencilla de aprender que paginación por cursor.

12. Desarrollo y producción

Durante el desarrollo:

React: <http://localhost:5173>

Express: <http://localhost:3000>

Postgres: localhost

React solicitará:

<http://localhost:3000/api>

En producción:

<https://gestor-documental.onrender.com/>

<https://gestor-documental.onrender.com/api>

Express servirá:

- La API bajo /api.
- El frontend compilado para las demás rutas.
- La ruta de React funcionará incluso al refrescar una página interna.

13. Primer bloque de implementación

Una vez aprobado este planteamiento, el primer bloque de código será limitado:

1. Crear backend/.
2. Crear package.json.
3. Crear la estructura inicial de carpetas.
4. Configurar ESLint.
5. Configurar .env.example.
6. Crear env.js.
7. Crear una aplicación Express mínima.
8. Añadir:
 - a. GET /api/health
 - b. manejo de rutas inexistentes
 - c. manejo central de errores
9. Añadir pruebas para /api/health.
10. Ejecutar lint y pruebas.

Todavía no incluirá:

- PostgreSQL.
- Supabase.
- Autenticación.
- Usuarios.
- Matters.
- Templates.
- Cambios al frontend.